# 박상천

## Software Developer

운영 환경의 문제를 분석하고

**시스템의 안정성과 효율성을 높이는** 개발자

Oracle • C# • ASP.NET • Java • Spring


010-3593-5474

pswlove38@naver.com

GitHub | SangCheonP




# ABOUT ME

## 운영 환경의 문제를 분석하고 성능 개선을 통해 시스템의 안정성과 효율성을 높이는 개발자, 박상천입니다.

Oracle 기반 데이터 처리 및 성능 개선 경험과 ASP.NET 기반 생산 시스템 개발 경험을 보유하고 있습니다. Docker·Jenkins를 활용한 배포 자동화 환경 구축 경험도 갖추고 있습니다.

## KEY STRENGTHS

### 문제 분석 & 개선
오류와 지연 로그 데이터를 상세히 역추적하여, 단순 수정을 넘어선 근본적인 해결책을 확립

### 성능 최적화
SQL 최적화와 데이터 처리 개선을 통해 안정적인 시스템 성능 확보

### 개발 경험
운영 환경 내 고가용성 및 복잡성 높은 생산 관리 프레임워크 개발에 활발히 기여

### 협업 & 소통
명확한 정보 공유와 열린 소통을 바탕으로 동료들과 함께 목표를 달성하는 시너지 창출

## EDUCATION

광운대학교 소프트웨어학부

4.01 / 4.5 | 2024.02 졸업

## CERTIFICATIONS

| | |
|---|---|
| 정보처리기사 | 2024.09 |
| SQLD (SQL 개발자) | 2024.06 |
| AWS Certified Cloud Practitioner | 2024.10 |
| 리눅스 마스터 2급 | 2025.01 |
| Toeic Speaking - IM2 | 2025.12 |
| JLPT - N3 | 2023.08 |

## AWARDS

- 특화 프로젝트 우수상 (SSAFY)
- 일본어 연극 대회 우수상

## TECH STACK

| | | |
|---|---|---|
| FRONTEND | React<br>JavaScript | · 생산 관리 시스템(MES)의 화면 설계 및 구현<br>· 사용자 편의성과 업무 효율을 고려한 UI 개발 |
| BACKEND | Java<br>Spring Boot<br>ASP.NET (C#) | · 제조·생산 시스템의 비즈니스 로직과 API 설계 및 개발<br>· 운영 환경의 요구사항을 반영하여 안정적인 서비스 구축 |
| DATABASE | Oracle<br>PostgreSQL | · Oracle 기반 대용량 데이터 처리 및 조회 성능 개선<br>· SQL 최적화와 데이터 처리 개선을 통한 응답 속도 향상 |
| INFRA | Docker<br>Jenkins<br>AWS (EC2) | · Docker와 Jenkins를 활용한 배포 자동화 환경 구축<br>· CI/CD 파이프라인 운영을 통한 배포 안정성과 효율성 향상 |

## WORK EXPERIENCE

### 노리시스템

**Software Developer** 2025.03 ~ 현재

- **삼성 반도체 품질 시스템(SSPS) 개발 및 유지보수**
  신규 기능 개발 및 기존 기능 개선을 통해 안정적인 서비스 운영에 기여
- **Oracle SQL 및 프로시저 개발**
  데이터 연계(I/F) 프로시저 개발 및 조회 성능 개선을 위한 SQL 최적화 수행
- **데이터 적재 성능 개선**
  SQL 최적화를 통해 데이터 적재 시간을 **3분**
  → **1분 20초**로 **약 55% 단축**
- **외부 고객사 MES 프로젝트 수행 ( MCS MES / 2026.06 ~ )**
  고객사 요구사항 분석부터 화면, API 개발 수행

# 프로젝트 개요

## SSPS

### (Samsung Semiconductor Production System)

반도체 생성 공정의 생산 현황, 수율 및 Lot 데이터를
모니터링하고 시각화하여 생산 운영자의 의사결정을 지원하는
생산 모니터링 시스템

**기간**
2025.03 ~ 진행중

**역할**
Full Stack Developer

**인원**
9명

**고객사**
삼성전자

### 기술 스택

- ASP.NET (C#)
- JavaScript
- PL/SQL
- Oracle 11g
- RealGrid2
- ECharts
- HTML/CSS

### 주요 성과

- ✓ 쿼리 최적화를 통해 데이터 적재 시간 3분 -> 1분 20초로 단축 (약 55% 개선)
- ✓ 생산 Dashboard 및 기준정보 화면 8종 신규 개발
- ✓ 운영 이슈 10건 이상 해결하여 시스템 안정성 향상
- ✓ 현업 요구사항을 반영한 기능 개선 및 유지보수 수행

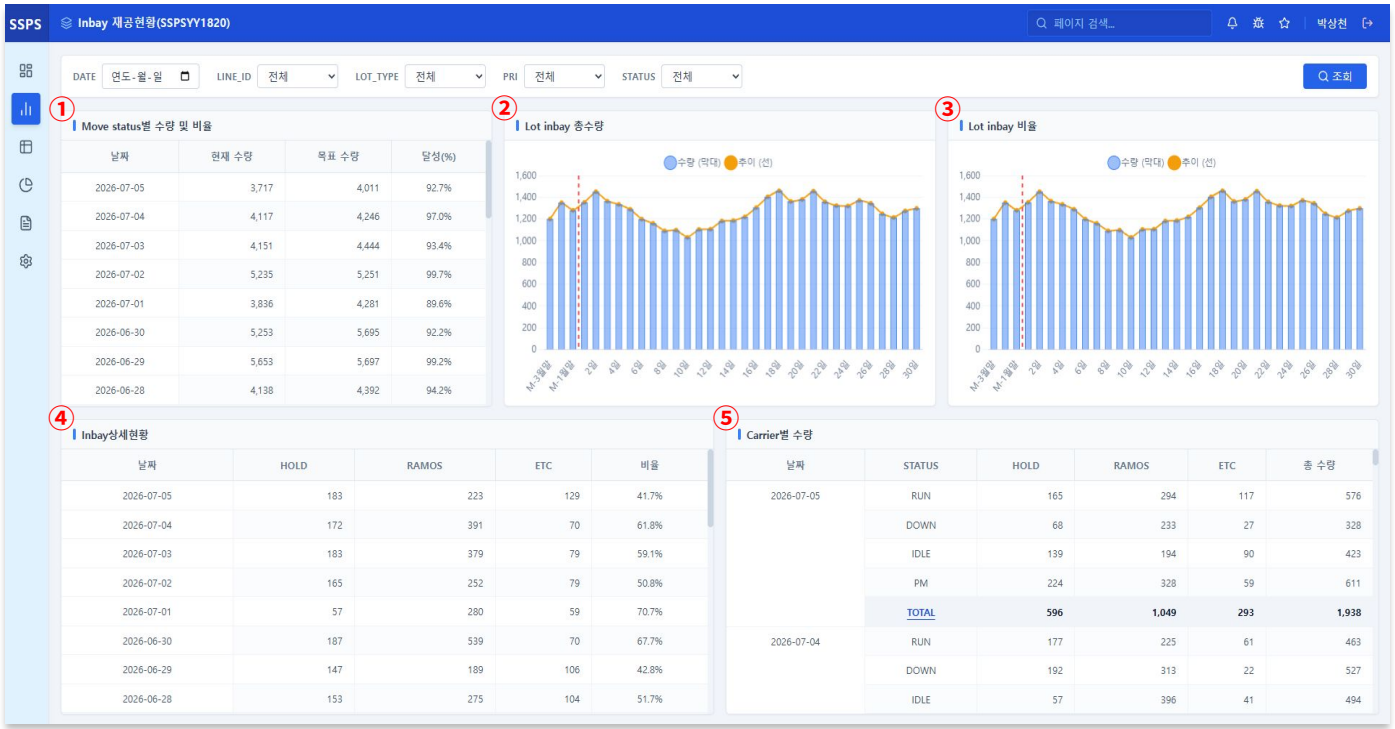
### 주요 담당 업무

**생산 현황 모니터링 화면 개발**
- 생산 현황 Dashboard 화면 개발
- 기준정보(CURD) 관리 화면 개발
- 데이터 Validation 및 예외 처리

**Oracle DB 개발 및 성능 개선**
- Oracle Procedure 개발
- SQL 튜닝을 통한 조회 성능 개선
- 대용량 데이터 조회 최적화

**운영 개선 및 유지보수**
- 운영 이슈 분석 및 장애 대응
- 사용자 요구사항 반영 및 기능 개선
- 배포 및 운영 안정화 지원

### 프로젝트 화면




① **Move status별 수량 및 비율**
→ 생산량 및 수율 현황 조회

② **Lot Inbay 총수량**
→ 공정별 Lot 현황 모니터링

③ **Lot Inbay 비율**
→ 공정별 비율 분석

④ **Inbay 상세 현황**
→ 상세 데이터 조회

⑤ **Carrier 별 수량**
→ Carrier 상태 모니터링

# SQL 튜닝 사례 Inbay 적재 성능 개선 (JOIN 구조 및 조회 로직 최적화)

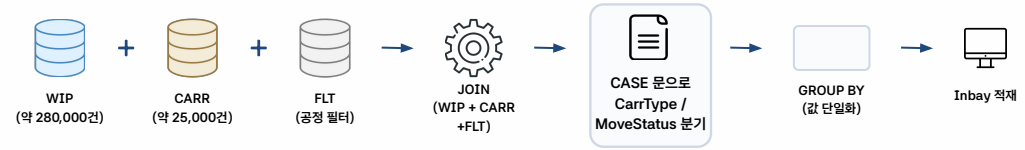
### 개요
Inbay 데이터 적재 프로세스에서 FLT JOIN 제거, CASE 분기 단순화,
GROUP BY 제거를 통해 SQL 처리 로직을 최적화하고
적재 성능을 개선하였습니다.

### 데이터 규모
- WIP : 약 280,000건
- CARR : 약 25,000건
- FLT : 공정 필터 정보

**Inbay 적재 시간**
**3분 → 1분 20초**
**▼ 약 55% 단축**

- ✓ FLT JOIN 제거로 JOIN 처리 비용 감소
- ✓ CASE 분기 단순화 및 GROUP BY 제거로 연산 최소화
- ✓ 적재 시간 3분 -> 1분 20초 (약 55% 단축)

## 1. 기존 구조(Before)




**문제점**
- ⊗ FLT JOIN으로 JOIN 처리 비용 증가
- ⊗ CASE 분기 로직으로 조건 비교 연산 증가
- ⊗ MAX() 및 GROUP BY 수행으로 집계 비용 증가
- ⊗ 대용량 데이터 처리 시 적재 시간 지연

**특징**
- 3개 테이블 조인 (WIP, CARR, FLT)
- CASE 문으로 조회 대상 컬럼 구분
- 조건 비교 후 컬럼 값 결정
- MAX() 및 GROUP BY를 통한 데이터 단일화
- 불필요한 연산으로 적재 성능 저하

## 2. 개선 구조 (After)




**개선 내용**
- ✓ FLT JOIN 제거
- ✓ CASE 분기 로직 단순화
- ✓ 조건절을 통한 직접 값 조회
- ✓ MAX() 집계 제거
- ✓ GROUP BY 제거

**기대 효과**
- ✓ FLT JOIN 제거로 JOIN 처리 비용 감소
- ✓ CASE 분기 단순화로 조건 비교 연산 감소
- ✓ MAX() 및 GROUP BY 제거로 집계 연산 최소화
- ✓ 적재 시간 3분 → 1분 20초 (약 55% 단축)

## 3. SQL 비교 (핵심 로직)

**Before (기존 SQL 일부)**

```
SELECT  lot_id,
❶       MAX(carr_type)     AS carr_type,
        MAX(move_status) AS move_status
  FROM (
        SELECT  lot_id,
❷               CASE WHEN category = 'CARR' AND ··· THEN carr
                END AS carr_type,
❸               CASE WHEN category = 'MOVE' AND ··· THEN move
                END AS move_status
          FROM (
                SELECT ···
                  FROM wip w
                 INNER JOIN carr c ON w.carr_id = c.carr_id
                 INNER JOIN flt f     ON c.flt_id = f.flt_id
          )
❹) GROUP BY lot_id;
```

**After (개선 SQL 일부)**

```
SELECT  lot_id,
❶       CASE WHEN ··· THEN ··· END AS carr_type,
        CASE WHEN ··· THEN ··· END AS move_status
  FROM  wip w
❷INNER JOIN carr c
        ON w.carr_id = c.carr_id
❸WHERE . . .;
```

튜닝 포인트 요약

| | |
|---|---|
| FLT JOIN 제거 | JOIN 처리 비용 감소 |
| CASE 분기 로직 단순화 | 조건 비교 연산 감소 |
| 조건절 직접 적용 | WHERE 절에서 직접 필터링 |
| MAX() 집계 제거 | 집계 연산 제거 |
| GROUP BY 제거 | 적재 시간 약 55% 단축 |

## 4. 성능 비교

| 항목 | Before (기존) | After (개선) | 개선 효과 |
|---|---|---|---|
| 조인 테이블 수 | 3개 (WIP, CARR, FLT) | 2개 (WIP, CARR) | ▼ 33% 감소 |
| CASE 로직 | 중첩 CASE | 단순 CASE | 조건 비교 단계 감소 |
| GROUP BY | 사용 | 제거 | 집계 연산 제거 |
| 적재 시간 | 약 3분 | 약 1분 20초 | ▼ 약 55% 단축 |

# 운영 이슈 해결 사례

## 제품별 공정정보 중복 적재 문제 해결

### 문제 상황

**제품별 공정정보 DIV 데이터 중복 적재 발생**

- 공정정보 수정 시 기존 DIV 데이터가 삭제되지 않음
- 기존 DIV 데이터 삭제 실패로 신규 데이터가 중복 적재
- 실적 및 현황 지표에 영향 발생

### 원인 분석

**DELETE 조건 변경**

- 삭제 조건을 org_step_id, org_line_id 기준으로 변경
- 기존 DIV 데이터에는 org_* 값이 없어 DELETE 대상에 포함되지 않음
- 기존 데이터가 남은 상태에서 INSERT 수행되어 중복 발생

### 해결 방법

**삭제 방식 수정 및 데이터 정리**

- 삭제되지 않은 DIV 데이터 식별
- 운영 반영 후 중복 적재 데이터 식별 및 삭제
- 삭제 조건을 PK (prod_id, site_id, step_id, line_id) 기준으로 원복

### 개선 효과

- ✓ 데이터 정합성 확보 및 중복 방지
- ✓ PK 기반 삭제로 삭제 성능 및 안정성 확보 (org_* 컬럼은 비PK, 참고용 값)
- ✓ 운영 안정성 및 유지보수성 향상

**Before (문제 발생)**

```
DELETE FROM b_rs_prod_div
WHERE prod_id      = :prod_id
  AND site_id      = :site_id
  AND org_step_id  = :org_step_id
  AND org_line_id  = :org_line_id
```

✗ 기존 DIV 데이터에 org_* 값이 없어 DELETE 실패
-> 기존 데이터가 남은 상태에서 INSERT 수행
→ DIV 데이터 중복 적재 발생

**After (개선 후)**

```
DELETE FROM b_rs_prod_div
WHERE prod_id  = :prod_id
  AND site_id  = :site_id
  AND step_id  = :step_id
  AND line_id  = :line_id
```

✓ PK 기준 DELETE 수행
-> 신규 DIV 데이터 정상 적재
-> 데이터 정합성 확보

### 처리 프로세스

1. **MASTER 공정정보 저장** (변경된 공정정보 저장)
2. **기존 DIV 데이터 삭제** (기존 분리 데이터 삭제)
3. **STEP_ID / LINE_ID 분리** (모든 경우의 수로 데이터 분리)
4. **DIV 데이터 적재** (분리된 데이터 저장)

**데이터 특성**

- MASTER 테이블에 step_id, line_id는 콤마(,) 문자열로 저장
- 나머지 컬럼 값은 그대로 사용
- step_id, line_id를 모든 조합으로 분리
- 분리된 데이터를 DIV 테이블에 적재

### 대상 데이터

- 테이블 : b_rs_prod_div
- 운영 환경 제품별 공정정보 데이터
- 해당 기간 중 중복 데이터 수 : 약 2배 발생

### 적용 내용 요약

- DELETE 조건 변경으로 기존 DIV 데이터 삭제 실패
- 운영 적용 후 중복 DIV 데이터 후처리
- 삭제 조건을 PK 기준으로 원복
- 데이터 정합성 및 운영 안정성 확보

### 주요 성과

- ✓ DIV 데이터 중복 적재 문제 해결
- ✓ 데이터 정합성 확보
- ✓ 운영 안정성 향상

# 프로젝트 회고 이번 프로젝트를 통해 얻은 경험과 성장

**성능 개선**과 **데이터 정합성**을 함께 고려하며, **운영 환경에서 발생하는 문제를 해결**하는 경험을 쌓았습니다.



## 기술적 성장

- SQL 구조 단순화를 통한 성능 개선 경험
- JOIN, CASE, GROUP BY 제거를 통한 SQL 튜닝 경험
- Oracle Procedure 및 대용량 데이터 처리 경험
- Before / After 비교를 통한 성능 개선 효과 검증

## 문제 해결 경험

- 운영 데이터 중복 발생 원인 분석 및 해결
- DELETE 조건 변경으로 발생한 데이터 정합성 문제 해결
- 기존 데이터 정리 및 운영 데이터 복구 경험
- 재발 방지를 위한 삭제 로직 개선

## 협업 및 소통

- 사용자 요구사항 분석 및 기능 개선
- 운영 이슈 발생 시 원인 분석 및 대응
- 변경 사항 영향도 검토 후 배포 진행
- 현업 담당자와 지속적인 커뮤니케이션 수행

## 앞으로 개선점

- SQL 실행 계획 분석 역량 강화
- Oracle 성능 튜닝 경험 확대
- 대용량 데이터 처리 구조에 대한 이해 향상
- 운영 환경을 고려한 안정적인 설계 역량 강화

**프로젝트를 통해 얻은 성과**

SQL 실행 개선
**55% 단축**
Inbay 적재 프로세스

데이터 정합성 확보
**중복 데이터 해결**
운영 데이터 복구

운영 환경 개선
**PK 기반 삭제 로직 적용**
재발 방지

사용자 신뢰도 향상
**정확한 데이터 제공**
운영 품질 향상

# 기타 프로젝트 경험 SSPS 외 수행 프로젝트

SSPS 프로젝트 외에도 다양한 개발 프로젝트를 수행하며
실무 경험과 문제 해결 역량을 쌓았습니다.



01

## MES 프로젝트

| | |
|---|---|
| 기간 | 2026.06 ~ 진행중 |
| 역할 | Full Stack Developer |
| 기술 | Spring Boot, PostgreSQL, React, Java |

주요 내용

- MES WMS 기능 개발
- Spring Boot API 개발
- React 기반 화면 개발 및 기능 개선

02

## SSMART OFFICE

| | |
|---|---|
| 기간 | 2024.10 ~ 2024.11 |
| 역할 | Infra 구축 / 메신저 기능 개발 |
| 기술 | Docker, Jenkins, Spring Boot, React, MySQL |

주요 내용

- Docker 기반 서비스 인프라 구축
- Jenkins 기반 CI/CD 파이프라인 구축
- 사용자간 메신저 기능 개발

03

## S.F.D (Smart Factory Detector)

| | |
|---|---|
| 기간 | 2024.08 ~ 2024.10 |
| 역할 | Infra 구축 / AI 모델 학습 |
| 기술 | Docker, Jenkins, Spring Boot, React, MySQL |

주요 내용

- Docker 기반 서비스 인프라 구축
- Jenkins 기반 CI/CD 파이프라인 구축
- Inception-ResNet-v2 기반 불량 검출 모델 학습



다양한 프로젝트 경험을 바탕으로 성능, 데이터 정합성, 운영 안정성을 함께 고려하는 개발자로 성장하겠습니다.